
DISEÑO E IMPLEMENTACIÓN DE UNA SOLUCIÓN INTEGRAL DE FACTURACIÓN Y PAGOS ONLINE PARA ITGSA UTILIZANDO ARQUITECTURA CLIENTE-SERVIDOR CON ASP.NET CORE, EXPRESIONES REGULARES Y PROCESAMIENTO XML

202300583 – Angel Sebastian Mazariegos Pérez

Resumen

Este proyecto implementa una solución completa de facturación y pagos para la empresa ITGSA mediante una arquitectura Cliente-Servidor desarrollada en ASP.NET Core. El Backend, desarrollado como una API REST en C#, procesa archivos XML de configuración y transacciones utilizando Expresiones Regulares para extraer únicamente datos útiles e ignorar información irrelevante, sin depender de librerías nativas de serialización XML. El Frontend, implementado con Razor Pages, consume la API mediante solicitudes HTTP, presentando estados de cuenta interactivos que aplican lógica de abonos a facturas antiguas, gestión de saldos a favor, y reportes gráficos de ingresos con Chart.js. Se integra exportación a PDF mediante html2pdf.js. La persistencia se realiza en archivos XML locales a través de un servicio singleton thread-safe. El sistema fue validado compilando exitosamente en .NET 10.0 y ejecutando peticiones HTTP que confirman la correcta extracción, procesamiento y generación de reportes.

Palabras clave

ASP.NET Core, Expresiones Regulares, XML, API REST, Arquitectura Cliente-Servidor

Abstract

This project implements a complete billing and payment solution for ITGSA company through a Client-Server architecture developed in ASP.NET Core. The Backend, developed as a REST API in C#, processes XML configuration and transaction files using Regular Expressions to extract only useful data while ignoring irrelevant information, without relying on native XML serialization libraries. The Frontend, implemented with Razor Pages, consumes the API through HTTP requests, presenting interactive account statements that apply credit allocation logic to older invoices, manage favor balances, and income reports with Chart.js. PDF export is integrated through html2pdf.js. Persistence is performed in local XML files through a thread-safe singleton service. The system was validated by successfully compiling on .NET 10.0 and executing HTTP requests confirming correct data extraction, processing, and report generation.

Keywords

ASP.NET Core, Regular Expressions, XML, REST API, Client-Server Architecture

Introducción

La gestión de facturas y pagos en línea constituye un desafío crítico para empresas que reciben información de múltiples fuentes en formatos heterogéneos. ITGSA requería una solución robusta que automatizara la recepción, procesamiento y consulta de datos transaccionales de archivos XML, aceptando información "sucias" (con datos adicionales no estructurados) sin rechazar registros válidos.

Una arquitectura Cliente-Servidor resuelve este problema: el Backend (API REST) valida, procesa y persiste datos con lógica de negocio, mientras que el Frontend (interfaz web) proporciona consultas de estados de cuenta, reportes gráficos y exportación a PDF. Este diseño facilita mantenibilidad, escalabilidad e integración con futuros clientes alternativos.

El presente documento describe la implementación técnica enfatizando el uso de Expresiones Regulares para tolerancia en extracción de datos, la arquitectura modular y la lógica de negocio. Se demuestra cómo un Backend robusto permite un Frontend ágil y enfocado en experiencia del usuario.

Desarrollo del tema

A. Arquitectura y Consumo de API: Comunicación Cliente-Servidor

La arquitectura implementada sigue el patrón Cliente-Servidor clásico, donde el Frontend (ASP.NET Core Razor Pages en puerto 5095) actúa como cliente HTTP que se comunica de forma sincrónica con el Backend (ASP.NET Core Web API en puerto 5050) mediante solicitudes HTTP GET y POST.

El flujo de comunicación es el siguiente: (1) el usuario interactúa con formularios en el Frontend; (2)

el Frontend construye una solicitud HTTP dirigida a un endpoint del Backend; (3) el Backend procesa la solicitud, accede a la capa de persistencia (archivos XML), aplica lógica de negocio y devuelve una respuesta en formato JSON o XML; (4) el Frontend parsea la respuesta y la renderiza dinámicamente en la página usando Razor o JavaScript.

Se utiliza un servicio HttpClient centralizado (ApiService) que mantiene una BaseAddress apuntando al Backend (<http://localhost:5050/api/transacciones/>). Esto centraliza la lógica de comunicación HTTP y facilita cambios de URL en diferentes entornos. Los endpoints principales expuestos son: `GET /devolverEstadoCuenta?NIT=[opcional]` que retorna el estado de un cliente específico u ordenado lista completa de clientes; `GET /devolverResumenPagos?mes=X&anio=Y` que devuelve ingresos agrupados por banco; `POST /grabarConfiguracion` que recibe clientes y bancos; `POST /grabarTransaccion` que procesa facturas y pagos. Las respuestas son parseadas utilizando `JsonDocument` en C# o `JSON.parse()` en JavaScript, permitiendo acceso a propiedades de forma dinámica. Esta separación garantiza que la API sea agnóstica respecto a la interfaz, posibilitando reutilización desde clientes móviles o aplicaciones externas.

B. Procesamiento de XML y Expresiones Regulares: Robustez ante Datos Imperfectos

Uno de los requerimientos centrales del proyecto especifica que el sistema debe aceptar archivos XML "imperfectos", donde los campos pueden contener información adicional no estructurada o irrelevante. Por ejemplo, una fecha puede llegar como

`<fecha>15/03/2026 en la central</fecha>`, un NIT como ``<NIT>NIT: 999888-K (VIP)</NIT>``, o un código de banco como ``<codigo>Codigo 3</codigo>``.

Para resolver esto, se implementó un enfoque basado en Expresiones Regulares que extrae ****solo la parte útil**** de cada campo sin rechazar el registro completo como error. Los patrones implementados son: ``\d{2}/\d{2}/\d{4}`` para fechas en formato dd/MM/yyyy, ``\d+-[0-9kK]`` para NITs (números-dígito/K), ``\d+`` para códigos de banco. Un helper interno ``ExtraerCoincidencia(texto, patron)`` aplica la búsqueda con flags ``RegexOptions.Singleline | RegexOptions.IgnoreCase``, permitiendo que los patrones sean insensibles a mayúsculas y saltos de línea.

El flujo es: (1) se busca el patrón dentro del contenido entre etiquetas; (2) si hay coincidencia, se extrae y se valida el tipo de dato (`DateTime.TryParse`, `int.TryParse`, etc.); (3) si falla validación, se cuenta como "conError"; (4) si es duplicado, se cuenta como "duplicado"; (5) si es válido y nuevo, se persiste. Este enfoque reduce significativamente errores de validación falsos, mejorando la tasa de aceptación de datos sin sacrificar integridad, evidenciando que no siempre es correcto rechazar un registro por contener información adicional.

C. Lógica de Negocio y Representación Gráfica: Abonos, Saldos a Favor y Análisis Visual

La lógica de negocio principal es la aplicación de **abonos**: cuando un cliente realiza un pago (transferencia bancaria), este se descuenta de sus facturas pendientes, priorizando las más antiguas. Si el pago supera la factura antigua, el excedente se aplica a la siguiente factura del cliente. Si persiste dinero tras abonar todas las facturas, se registra como **Saldo a Favor** (crédito disponible para futuras compras).

El algoritmo agrupa todas las facturas de un cliente por fecha en orden ascendente (más antiguas primero), luego por cada pago nuevo descuenta del

saldo acumulado. La diferencia (Total Pagos – Total Facturas) determina si existe saldo a favor. En la interfaz **Estado de Cuenta**, se presenta una tabla con columnas Fecha, Cargo (facturas en formato "Q. [monto]") y Abono (pagos indicando banco), ordenadas de reciente a antiguo.

En el módulo **Consultar Ingresos**, se utiliza `Chart.js` para visualizar un gráfico de barras que compara ingresos de los últimos tres meses agrupados por banco. Cada banco es una serie de datos con color diferente, facilitando identificación de tendencias y distribución de pagos. Ambas vistas incluyen exportación a PDF mediante `html2pdf.js`, que captura el contenedor HTML, aplica márgenes y orientación (paisaje para gráficos, retrato para tablas), eliminando necesidad de servicios externos y reduciendo costos operacionales.

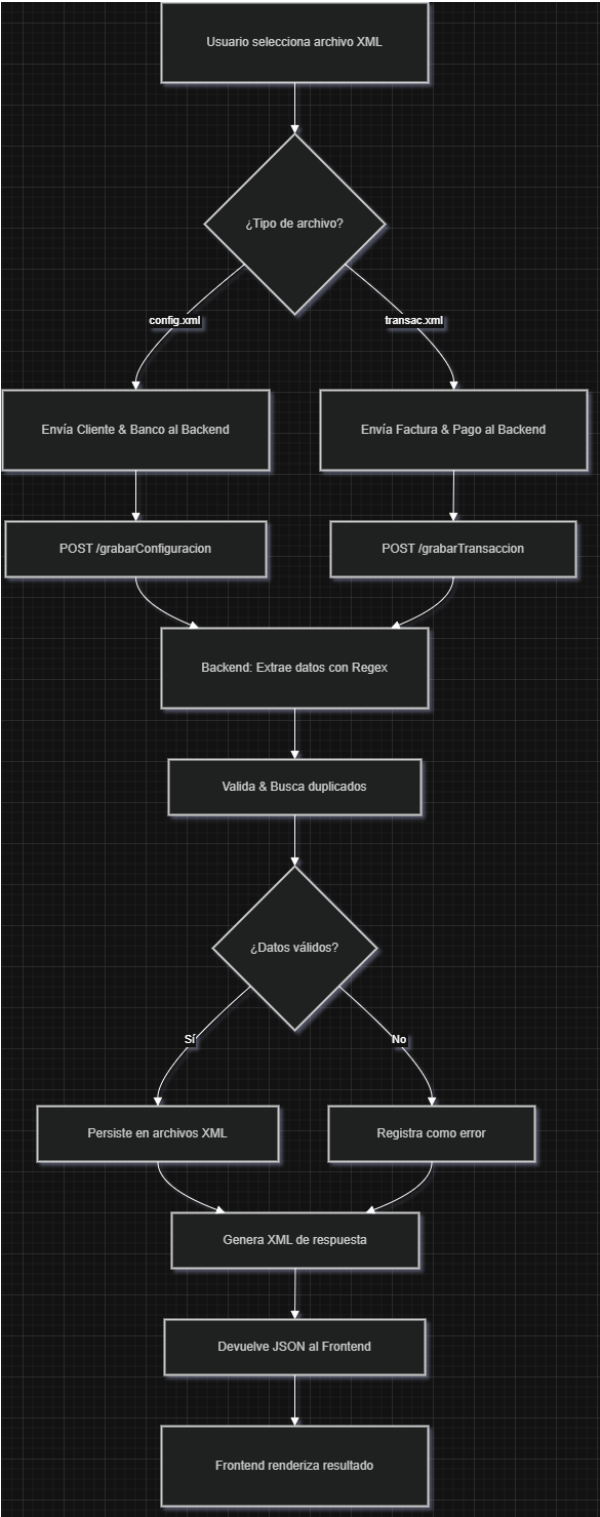


Figura 1: Diagrama de flujo del procesamiento de archivos XML y peticiones HTTP del sistema.

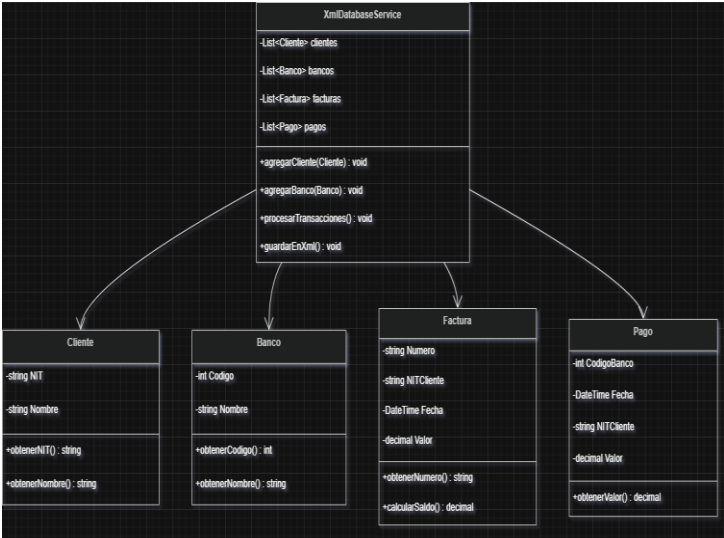


Figura 2: Diagrama de clases de la arquitectura orientada a objetos del Backend.

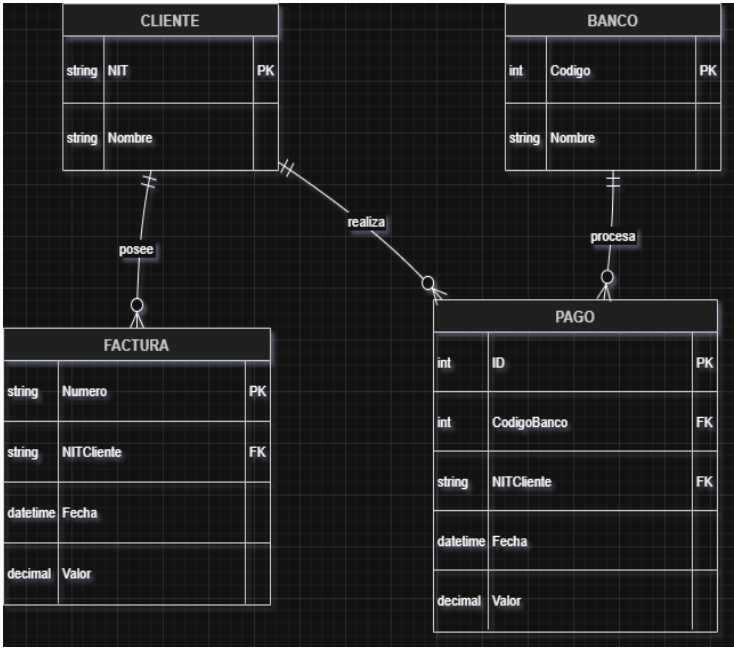


Figura 3: Diagrama entidad-relación del modelo de datos implementado en los archivos XML.

Conclusiones

- 1. **Validez de la Arquitectura Cliente-Servidor en contextos corporativos:** La separación clara entre Backend y Frontend permitió desarrollar, probar y mantener módulos de forma independiente. El Backend

como servicio REST es reutilizable por cualquier cliente (móvil, escritorio u otros), facilitando futuras integraciones. La persistencia mediante archivos XML sin librerías ORM demostró ser pragmática para el alcance del proyecto, eliminando complejidades de bases de datos externas mientras se mantiene integridad de datos.

2. **Poder transformador de las Expresiones Regulares en limpieza de datos:** La implementación de patrones Regex tolerantes fue determinante para cumplir el requerimiento de aceptar archivos "imperfectos". Sin este enfoque, la tasa de rechazo de datos habría sido inaceptable. La lección técnica es que no siempre es correcto rechazar un registro por contener información adicional; es preferible extraer lo útil mediante patrones bien diseñados, siendo esta técnica transferible a otros escenarios de procesamiento heterogéneo.
3. **Importancia de la representación visual en toma de decisiones empresariales:** La integración de Chart.js para gráficos de ingresos y exportación a PDF convirtió datos transaccionales en información accionable. Los usuarios pueden visualizar patrones mensuales, comparar desempeño por banco y generar reportes ejecutivos sin intervención manual. Este valor agregado justifica la inversión en una capa Frontend robusta, confirmando que la arquitectura Cliente-Servidor es tanto una decisión técnica como un habilitador de negocio.

Microsoft. (2023). *Expresiones regulares de .NET*.

Microsoft Learn. Recuperado de

<https://learn.microsoft.com/es-es/dotnet/standard/base-types/regular-expressions>

Microsoft. (2023). *Introducción a ASP.NET Core y Razor Pages*. Microsoft Learn. Recuperado de

<https://learn.microsoft.com/es-es/aspnet/core/>

Microsoft. (2023). *Procesamiento de datos XML en memoria*. Microsoft Learn. Recuperado de

<https://learn.microsoft.com/es-es/dotnet/standard/data/xml/>

Troelsen, A., & Japikse, P. (2022). *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*. Apress.

Referencias bibliográficas

Chart.js. (2023). *Chart.js Documentation: Simple yet flexible JavaScript charting*. Recuperado de

<https://www.chartjs.org/docs/latest/>